Database Design Analysis: Airline Reservation System

The nine-table schema is close to third normal form, but five flaws cause redundancy and anomalies: a repeated city and country, a repeated flight route, a derived seat total, seat status stored per aircraft instead of per flight, and booking status copied into three other tables.

1. Data Redundancy

Redundancy means the same fact is stored in more than one place, so every change must be repeated everywhere. The current schema has six such spots.

#	Table(s)	What is repeated	Why it repeats
1	Airport	Country (and City)	Every airport in the same country or city stores the same text again
2	Flight	Departure and arrival airport of a flight number	One flight number flies the same route every day, but each dated flight row repeats the route
3	Aircraft, Seat	Total_Seats	Equals the number of Seat rows for that aircraft, so it is stored twice
4	Seat, Booking	Seat_Status and Booking_Status	"Reserved" and "Confirmed" describe the same event in two tables
5	Booking, Ticket, Payment	Cancelled / Valid / Failed statuses	A cancellation must be written to three tables
6	Booking, Payment, Baggage	Fare, Amount and Excess_Baggage_Fee	Amount paid is normally Fare plus Excess_Baggage_Fee, so it is stored twice

Sample rows for the first two spots show how the repetition looks:

Airport_ID	Airport_Code	City	Country
1	BLR	Bengaluru	India
2	DEL	Delhi	India
3	BOM	Mumbai	India

Flight_ID	Flight_Number	Departure_Airport_ID 1	Arrival_Airport_ID	Departure_DateTime
101	AI502	1	2	2026-11-01 06:00 2026-
102	AI502	1	2	11-02 06:00 2026-11-03
103	AI502		2	06:00

"India" appears three times, and the route 1 → 2 appears three times. Over a year, AI502 alone would repeat its route 365 times.

2. Insertion, Deletion and Update Anomalies

Each anomaly below follows from a redundancy in Section 1.

Type	Table	Scenario	What goes wrong
Insertion	Airport	Record a new country or city before any airport exists there	Not possible: country and city exist only as text inside an Airport row
Insertion	Flight	Publish the route of new flight number AI777 before its first dated flight is scheduled	No row can hold the route without a date, aircraft and status
Insertion	Seat	Mark seat 12A available on a newly created flight	Seat_Status belongs to the seat, so there is no per-flight record to insert

Type	Table	Scenario	What goes wrong
Insertion	Passenger	Register the same person twice with the same Email	Allowed: Email is not UNIQUE, so one person gets two Passenger_IDs
Deletion	Airport	Delete the last airport in a country	The only record that the country or city existed is lost
Deletion	Flight	Purge old AI502 flights after the season	The route AI502 flew is lost with the last row
Deletion	Booking	Delete a cancelled booking	Its Payment and Ticket rows are blocked, or (with ON DELETE CASCADE) destroyed, along with the refund record
Deletion	Seat	Remove seat 12A after a cabin refit	Blocked by old bookings, or erases the history of who held that seat
Update	Airport	Fix the spelling of a country in one row	Other rows still hold the old spelling, so "India" and "INDIA" count as two countries
Update	Flight	AI502 moves from DEL to HYD	Every dated row needs updating; one missed row gives one flight number two routes
Update	Aircraft	Add 10 seats to the Seat table	Total_Seats stays at the old number, so capacity checks give wrong answers
Update	Seat, Booking, Ticket	Cancel a booking	Booking_Status changes, but Seat_Status stays "Reserved" and Ticket_Status stays "Valid"
Update	Seat	Book 12A on the 1 Nov flight	12A shows "Reserved" on the 2 Nov flight too, because the status is stored per seat, not per flight
Update	Booking, Payment	Fare is corrected on the booking	Payment.Amount keeps the old value, so the two disagree

Three integrity gaps make these worse:

- **Double booking is not blocked.** Nothing stops two confirmed bookings for the same Flight_ID and Seat_ID.
- **Seat and aircraft can mismatch.** Booking stores Flight_ID and Seat_ID separately, so a seat from a different aircraft than the flight's can be booked.
- **Cardinalities are not enforced.** The 1:1 links Booking–Ticket and Booking–Baggage are plain foreign keys, so a booking can get several tickets. Booking–Payment is documented as 1:1, yet Payment_Status includes "Failed", which implies retries and therefore 1:M. No foreign key has NOT NULL, so a booking can also exist with no passenger or flight.

3. Functional Dependencies

Only Airport and Flight break third normal form (3NF). The other problems come from derived data and one misplaced attribute, which normal forms alone do not catch. A functional dependency X → Y means each value of X determines exactly one value of Y. All nine tables hold only atomic values, so all satisfy first normal form (1NF); every key is a single column, so none has a partial dependency and all satisfy 2NF.

Table	Functional dependencies	Normal form
Passenger	Passenger_ID → First_Name, Last_Name, Email, Phone, Date_of_Birth, Gender. Intended but not enforced: Email → Passenger_ID	BCNF, if Email is made UNIQUE
Airport	Airport_ID → all other columns. Airport_Code → Airport_ID (an IATA code is unique). City → Country, so Airport_ID → City → Country is a transitive dependency	2NF; fails 3NF
Aircraft	Aircraft_ID → all other columns. Registration_No → Aircraft_ID. Total_Seats is also the count of that aircraft's Seat rows, so it is derived data	3NF, with a derived column

Table	Functional dependencies	Normal form
Flight	Flight_ID → all other columns. Flight_Number → Departure_Airport_ID, Arrival_Airport_ID (a flight number keeps its route), so Flight_ID → Flight_Number → route is transitive	2NF; fails 3NF
Seat	(Aircraft_ID, Seat_Number) → Seat_ID, Seat_Class. As written, Seat_ID → Seat_Status. In reality the dependency is (Flight_ID, Seat_ID) → Seat_Status	BCNF on paper, but Seat_Status is in the wrong table
Booking	Booking_ID → all other columns. Intended but not enforced: (Flight_ID, Seat_ID) → Booking_ID for active bookings	BCNF
Ticket	Ticket_ID → all other columns. Ticket_Number → Ticket_ID. Booking_ID → Ticket_ID if the link is 1:1	BCNF; Ticket_Status copies Booking_Status
Payment	Payment_ID → all other columns. Transaction_ID → Payment_ID. Amount is derived from Fare and Excess_Baggage_Fee	BCNF; Amount duplicates other data
Baggage	Baggage_ID → all other columns. Booking_ID → Baggage_ID if the link is 1:1. Excess_Baggage_Fee follows from Total_Weight and the booking's Seat_Class	BCNF; the fee is derived from other tables

Two assumptions sit behind this table. City names can repeat across countries (Birmingham, UK and USA), but the schema stores no state or region, so City → Country is assumed. And an airline keeps the same route for a flight number, which is why Flight_Number → route holds.

4. How Poor Design Leads to Anomalies

A fact stored in several places has no single owner, so the database cannot keep the copies equal; the application must remember every copy, and one missed copy is an anomaly. Poor design causes anomalies in four ways:

1. **Repeated facts** (country in Airport, route in Flight) cause update anomalies, because one change must reach many rows.
2. **Facts stored inside an unrelated row** (a country inside an airport, a route inside a dated flight) cause insertion and deletion anomalies, because the fact can only be added or removed together with that row.
3. **Facts in the wrong table** (Seat_Status on Seat) mean the real situation, "seat 12A is free on one date and taken on another", cannot be stored at all.
4. **Missing constraints** (no UNIQUE, no NOT NULL, no composite foreign key) let invalid data in, so double bookings and orphan-like rows are accepted silently.

Worked example: cancelling booking 5001 (seat 12A, flight 101). The application updates Booking_Status but misses the other copies:

Table	Column	Value now	Should be
Booking	Booking_Status	Cancelled	Cancelled
Ticket	Ticket_Status	Valid	Cancelled
Seat	Seat_Status	Reserved	Available
Payment	Payment_Status	Successful	Needs a refund state (no "Refunded" value exists)

The database accepts every one of these rows. The result is a cancelled booking whose ticket is still valid, a seat that cannot be resold, and, because Seat_Status is stored per seat, seat 12A also appears reserved on flights 102 and 103 that nobody booked.